ADR-002 — The v1 store, and how the server orders songs

- **Status:** Accepted
- **Date:** 2026-09-05
- **Deciders:** Jay
- **Supersedes:** none. Extends ADR-001, which chose Go and the sync-first shape.

Context

Phase 2 needs somewhere to keep a scanned library and something to order it by. Two questions had to be settled before the HTTP API could be written, and both have answers that are easy to get wrong quietly.

1. **Where does the catalog live?** ADR-001 promised "no database server required for v1". That says what we will not do, not what we will.
2. **In what order does a browse request return songs?** A server that sorts differently from the client produces a list the player cannot recognise, and nothing about it looks broken — the songs are all there, in an order that is merely wrong.

Decision

1. The catalog is an in-memory index, rebuilt at start

`internal/library` walks the library once at startup, holds every entry in memory, and serves every request from that. There is no database, no file format, and nothing to migrate.

Why: the whole catalog for a large library is small. The corpus indexes in 15 ms at 22 cases and 14 ms at 23; even a library of tens of thousands of songs is tens of megabytes of struct and a scan measured in seconds, on a Pi as much as on a workstation. Against that, a persisted catalog buys

a faster start and costs a schema, a migration path, and a second source of truth that can disagree with the disk. Every one of those is a place for the server to be confidently wrong about what it holds.

What it costs, stated plainly: start-up time is proportional to library size, and a song added to the library is invisible until a rescan. Both are acceptable for v1 and both are fixable underneath this interface — `library.Store` already swaps a freshly built index in atomically, so a rescan never shows a half-built library to a request in flight.

Rejected: SQLite. The obvious choice, and the wrong one here for a specific reason: the cgo driver would break the cross-compilation that ADR-001 rests on — `linux/arm64` for the Pi from a Windows workstation is one `GOOS` / `GOARCH` away right now and would stop being so. The pure-Go driver avoids that but is a large dependency to carry for a query load of "filter a slice and sort it".

2. Packed archives are cached on disk, keyed by package hash

A song found as anything but a `.sng` — a loose folder, a `.zip` or a `.7z` — has to be packed before it can be served. `internal/packcache` packs it once into `<data>/packs/<package_hash>.sng` and serves the file thereafter. Packing is deterministic and identical across the three shapes, so an evicted entry re-packs to the same bytes and the cache is a pure accelerator, never data.

Why a file rather than streaming to the response: streaming means no `Content-Length`, no `Range`, and therefore no resume. A sync client that cannot resume a 40 MB song over a home wifi link is a sync client that starts again from zero, and Phase 2b is exactly that client.

The package hash is a sound key because it is a hash of the content: the same package produces the same archive, on any server, after any rescan. Archives are written to a temp file and renamed, so a crash or a full disk mid-pack cannot leave a truncated archive at the final name for every later request to serve as though it were whole.

3. Sorting reproduces the client's `SortString`, exactly

`internal/sortkey` is a faithful reimplementation of YARG.Core's `SortString` and the three transforms behind it, and `internal/library` orders by the twelve `SongAttribute` values using upstream's own comparer chains.

Why go to that trouble: the normalisation is not cosmetic. It drops a leading article, so "The Beatles" files under B. It folds diacritics, so "Björk" sorts beside "Blur" rather than after every ASCII name in the library. It strips Unity rich-text markup, so a chart titled `<color=#ff0000>Red</color>` does not file under `<`. A server that skipped any of this would return a list that is internally consistent and unlike anything the player sees in the game.

Three upstream behaviours are reproduced although each looks like a defect, because agreeing with the client is the point and "improving" it would mean disagreeing:

- Only uppercase `Æ` is expanded to `AE`; lowercase `æ` is untouched and sorts as non-ASCII.
- Comparison is by UTF-16 code unit, not code point, so an emoji sorts below `U+E000`.
- The article list is `the, el, la, le, les, los` — **not** `a` or `an`.

4. Two documented divergences from the client

Neither is a mistake and both are recorded here so they are not quietly "fixed" later.

The final tie-breaker is `package_hash`, not the entry's location. Upstream breaks a tie with `SortBasedLocation`, an absolute path on the player's own machine. That value would make the same library sort differently on two machines, and the server has no such path for a client in any case. `package_hash` is stable, content-derived and unique per package.

Genre, Subgenre, Source, Artist_Album and DateAdded have no comparer upstream — they are collected as grouping values rather than ordered. The chains used for those five are ours, and they are ours in the code comments too, rather than presented as parity.

5. Search matching is ours; only the folding is the client's

A query is folded through the same `SortString` normalisation, so a player who cannot type "Björk" still finds it. What happens next — a substring test across the eight text attributes — is this server's own. Upstream's own search logic has not been read, so no claim of parity is made for it, and the API documentation says so.

Consequences

Good

- No schema, no migration, no second source of truth about what the library contains.
- Cross-compilation to every promised platform is untouched; the only new dependency is `golang.org/x/text`, which is pure Go, for Unicode normalisation that the standard library does not provide.

No longer true as of 2026-09-06. `github.com/bodgit/sevenzip` was added for `.7z` ingest, taking the compiled third-party package count from 3 to 71 and the binary from 9.92 MB to 11.62 MB. Cross-compilation is unaffected — every dependency is pure Go and all six targets still build with `CGO_ENABLED=0` — but "the only new dependency" is a sentence about a state this project has left. See ADR-003. - Browse order matches the game, which is the difference between a useful list and a correct-looking one. - A packed archive is cached once and is byte-identical on every later request, which is what makes `ETag` and `Range` honest.

This was written as a design intent and shipped as a defect. `PackDir` drew a fresh random mask on every call, so the property held only while the cache held the archive — the moment one was evicted, the next request produced different octets under the same strong `ETag`. Found on

2026-09-06 by syncing two machines from one server and comparing SHA-256s: **16 of 22 archives differed**, 16 being exactly the number the bounded cache had re-packed. Fixed by deriving the mask from the package hash (`sng.MaskKeyFor`). The bullet is now true, and it is now measured rather than intended.

Bad / accepted

- Start-up cost grows with the library, and a new song needs a rescan to appear.
- Memory holds the whole catalog. Acceptable at v1 scale; the first library that makes it hurt is the signal to put something underneath `library.Store`, not to change the API.
- ~~The pack cache grows without a bound or an eviction policy.~~ **Closed 2026-09-05.** It is bounded by `pack_cache_max` (default 2 GiB) with LRU eviction. The consequence this entry understated: an archive is within 2% of the size of the folder it came from — measured, 225,406 bytes of library produced 229,515 bytes of cache — so "grows without a bound" meant *a second copy of the whole loose library*, which is a Pi's entire SD card rather than an untidiness. The content-keyed property is what makes eviction free, exactly as this entry said: an evicted archive is rebuilt byte-identically and its package hash is unchanged. **That last sentence was false for one day.** It was true of the *package hash*, which is computed from the folder, and simply assumed of the *archive*, which was not — see the correction above. Deriving a property of the output from a property of the key is the mistake; only the bytes settle it.
- Reproducing `SortString` means it can drift from upstream, exactly as ADR-001 accepted for the format parsers. Mitigated the same way: by measuring against a real client.

Concurrency, measured 2026-09-07 — and the defect it found

Everything above was measured with one client at a time. "A Pi on the LAN serves a shared library" is inherently concurrent, so this is what several clients at once actually do.

As with archive ingest, a throwaway probe **printed** what happened before anything was asserted. It found one real defect, confirmed two properties, and corrected one promise this ADR had been making loosely.

The defect: a 404 for a song that exists

The handler asked the cache for a **path** and then opened it as a separate step. An evicting goroutine could remove the archive in between, and the server then answered

> **404** — *this song is no longer where the index says it is; rescan*

for a song that was perfectly present. Confidently wrong, and it points the operator at a library that is fine.

Measured on Windows, 64 clients pulling a 40-song library through a cache bounded to two archives: **2, 1 and 0 spurious 404s across three runs of 2,560 requests.** Rare enough never to be seen by hand, common enough to happen constantly on a busy server.

`packcache.Open` now returns an **open file** rather than a path, so there is no moment when the caller holds a name but not a handle. On POSIX an unlinked file stays readable through an open descriptor; on Windows the remove fails while the handle is open and `enforceBound` already skips that entry. `Path` remains, implemented on top of `Open`, for tests and tools that are not serving concurrently.

A `.sng` already in the library is still opened from its own path, because there the 404 means what it says: someone really did move the file.

The second defect, which the first fix left behind — and which only Linux showed

Handing back an open file closed the *caller-side* window. It left a smaller one inside `packcache.openOnce`: the archive is packed to a temp file, renamed into place, and only then opened, and between the rename and the open it is an ordinary cached entry that any other goroutine's `enforceBound` may remove.

This window was written up here and in the code as *real by inspection but never reproduced*, on the strength of a stress build with the retry disabled passing **five runs out of five**. Every one of those runs was on Windows, **where an open handle blocks `os.Remove` and the platform masks the race**. A Windows-only negative was recorded as a fact about the code — the same error this project has now made three times, and the reason the standard is *verify by measuring*.

Linux disagreed on the first pipeline that ran the eviction test:

> `1 of 7680 requests failed under eviction (0 of them transport-level); [ca43baef: status=500 bytes=37]`

A genuine server 500 for a song that exists. Reproduced immediately afterwards at the `packcache` level on Linux — **8 of 3,456 packs lost their archive in that window** — which is about a 0.2 % chance per pack here and evidently several percent on a CI runner, since three consecutive losses is what it takes to reach a client and CI reached one in 7,680.

The fix holds the eviction lock across the rename and the open. `enforceBound` is the only thing that removes a `.sng` and it holds the same lock for its whole body, so the window is closed by construction rather than narrowed. Lock ordering is shard-then-evict and never the other way, so there is no cycle.

Two things were added so this cannot quietly come back:

- `Cache.Vanished()` counts every time a freshly packed archive is gone before its packer can open it. It is asserted to be **zero**, which turns a once-in-a-pipeline flake into a number. On the broken code the same test reports 8 in 3,456; on the fixed code, 15 consecutive runs report 0.
- **The concurrency tests now give the server a logger and print what it logged.** The CI failure said `status=500 bytes=37` and nothing else, and *two* different 500s on that path have bodies of exactly 37 bytes — "could not pack this song" and "could not read this song" are both 24 characters. Which one it was had to be inferred. The server knew and was throwing it away.

The retry (`openAttempts = 3`) is kept as a second line, but it is no longer the mechanism: it guards against a failure mode nobody has thought of yet, and `Vanished()` reports if it ever fires.

Two properties that held

- **The thundering herd collapses.** 64 concurrent requests for one *uncached* song produce one pack, one archive in the cache, and 64 byte-identical responses. The sharded lock and the double-check after acquiring it do what they claim.
- **Eviction under load costs a re-pack and never data.** 7,680 requests across 64 clients against a cache bounded to a **single** archive: every response byte-identical to the same song packed serially. Zero mismatches. This is the claim at the top of `packcache` that was false once before, now measured under the conditions most likely to break it. What that run did **not** cover is a request failing outright: it was a Windows run, and the section above is what Linux found in the same test.

One promise corrected: the bound is a high-water target, not a hard cap

`pack_cache_max` is enforced *after* each insert, so concurrent packs can finish before any eviction runs. Peak cache observed at **1.5× the bound with 4 and 16 clients, 2.25–2.75× with 64** — an overshoot of one to three archives, growing with concurrency rather than unbounded.

On Windows there is a second contributor, and it is deliberate: a file being served cannot be evicted, because the handle that guarantees the song is servable is the same handle that blocks the remove. **That is the right way round.** The bound is a disk-space target; serving a song that exists is a correctness promise, and when they conflict the promise wins.

Operators sizing a cache on a small SD card should therefore treat `pack_cache_max` as "roughly this, plus a few archives under load", not as a ceiling.